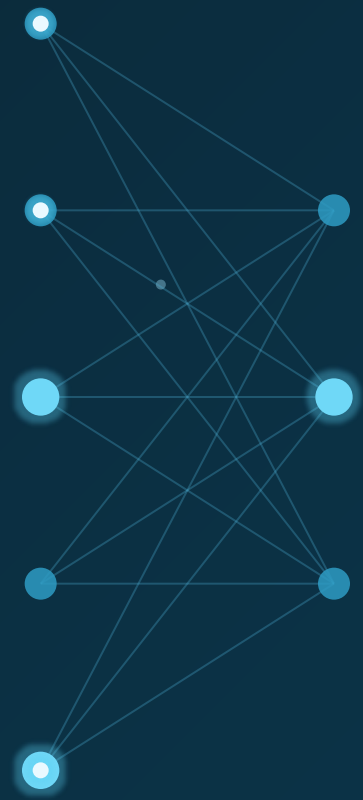


Introduction to Machine Learning

Furkan Cantürk



Course Roadmap

Day 1: Foundations

- Introduction to ML
- Regression and classification
- Confusion matrix, performance metrics and imbalanced data
- Overfitting and cross validation
- Models: Linear Regression, Logistic Regression, Naive Bayes, kNN, decision trees

Day 2: Advanced Topics

- Ensemble models: bagging, boosting, stacking
- Deep learning (brief)
- Unsupervised learning: clustering and PCA
- Forecasting
- Hyperparameter optimization, hands-on with Optuna and MLflow
- Data science steps, ML lifecycle, interpretable ML

PRE-LECTURE QUIZ

Introduction to Machine Learning

ff-quizzes.netlify.app/en/ml/quiz/1

What Is Machine Learning?

Traditional programming

Data + Rules → Answers

A developer writes the rules by hand.

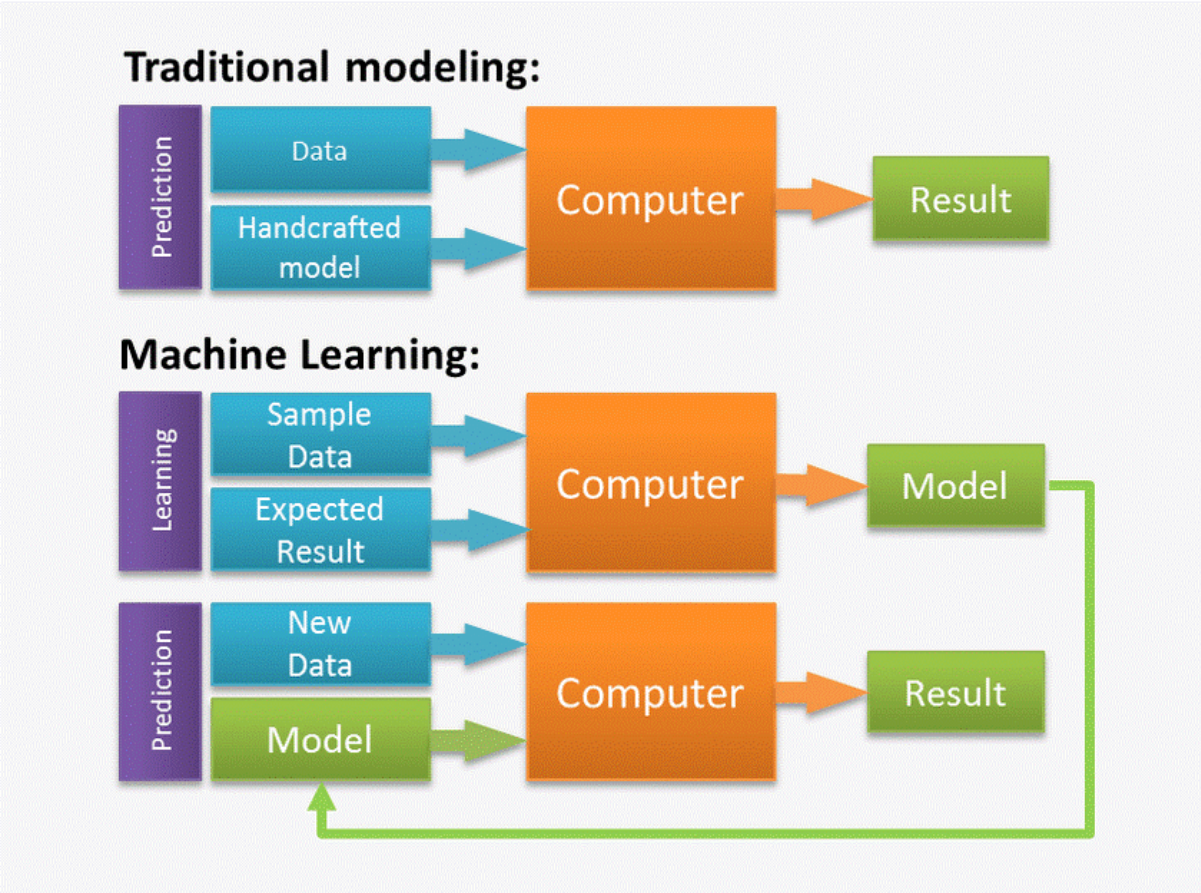
Machine learning

Data + Answers → Rules (the model)

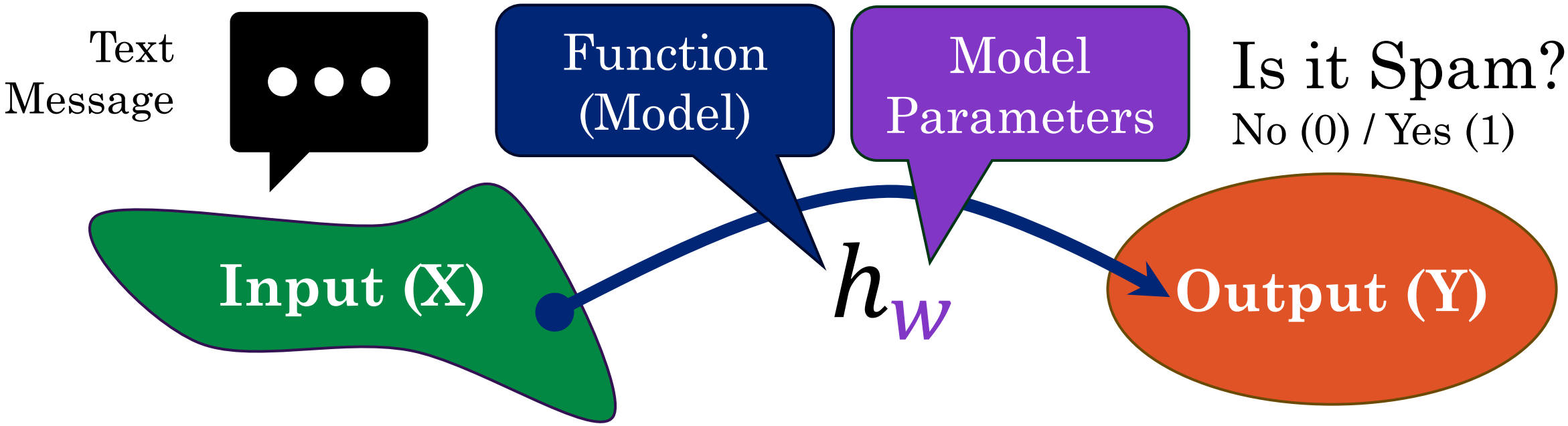
The algorithm learns the rules (patterns) from examples, then predicts answers for new data.

Everyday examples

- Spam filters in email
- Movie and product recommendations
- Fraud detection on card payments
- Sales and demand forecasting



Machine Learning as **Learned Function Approximation**



Machine learning becomes the process of **"learning"** the **model parameters** from data or interaction with the world.

Slide taken from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Linear Regression

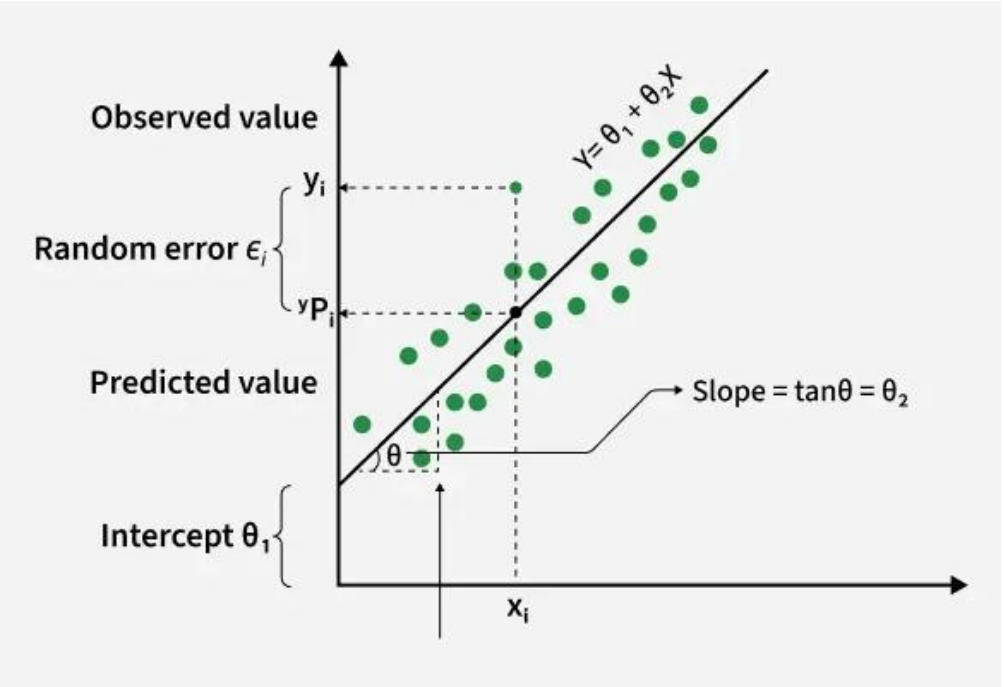
The model

$\hat{y} = w_0 + w_1x_1 + \ldots + w_nx_n$

w_0 is the intercept; each weight w_i shows how much feature x_i moves the prediction.

Example: Price ($K) = 40 + 2.2 × Size ($m^2$).

A 120 m^2 house is predicted at $304K.



Training

Find the weights that minimize the squared error between the predictions ŷ and the true values y.

Strengths and limits

- Fast, simple and easy to interpret
- Assumes a linear relationship; sensitive to outliers

A Dataset

Features (X)				Label (y)
District	**Size**	**Rooms**	**Age**	**Price**
Central	120 m^2	3	10 yrs	$320K
North	85 m^2	2	25 yrs	$190K
Central	150 m^2	4	5 yrs	$410K
South	95 m^2	2	40 yrs	$160K
North	200 m^2	5	2 yrs	$480K

In code, **X** is the feature matrix (n_samples × n_features) and **y** is the label vector.

Sample

One row: one **observation** (one house)

Feature

An input **variable** the model uses (size, rooms, district, age)

Label / Target Variable

The value we want to predict (price)

Preparing Data

Missing values

- Drop rows or columns with too many gaps
- Impute with the mean, median or most frequent value

Categorical encoding

- One-hot encoding for unordered categories (District → North, South, Central)
- Ordinal encoding for ordered categories (Low < Medium < High)

Feature scaling

- Standardization: mean 0, standard deviation 1
- Min-max scaling: values rescaled to 0–1
- Needed for kNN, linear models and gradient descent; trees don't need it

Feature engineering

- Create new features from existing ones (price per m^2, house age)
- Extract parts of dates (month, day of week)

In scikit-learn: SimpleImputer, OneHotEncoder and StandardScaler, combined in a ColumnTransformer and / or Pipeline

Data Leakage

Information from the validation or test data, or from the future, reaches the model during training. **The score looks better than real-world performance.**

Common causes

- Scaling or imputing on the full dataset before splitting
- Features that are unknown at prediction time
- Random split on time series: training on the future
- Duplicate rows in both training and test sets

Wrong: scaler sees the test rows

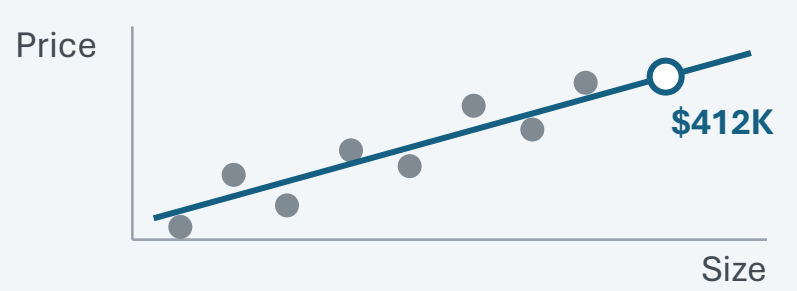
```
scaler = StandardScaler().fit(X)
X_s = scaler.transform(X)
X_tr, X_te, y_tr, y_te =
    train_test_split(X_s, y)
```

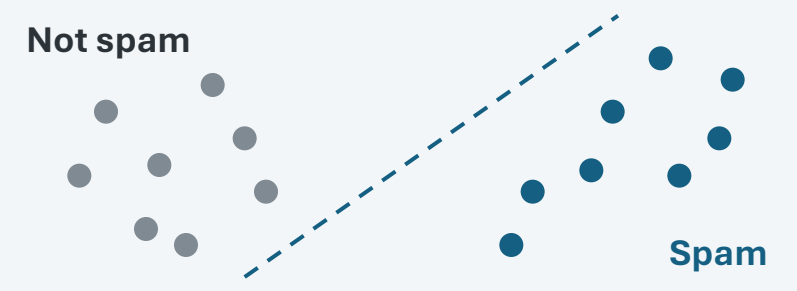
Right: split first, fit on training data only

```
X_tr, X_te, y_tr, y_te =
    train_test_split(X, y)
model = make_pipeline(
    StandardScaler(), LogisticRegression())
model.fit(X_tr, y_tr)
```

Regression vs Classification

	Regression	Classification
Output	A continuous number	A category (class)
Example	What will this house sell for? → $412K	Is this email spam? → Spam, 92% probability
Real-world uses	Sales forecasts, delivery times, demand	Fraud detection, diagnosis, image tagging
Metrics	MAE, MSE, RMSE, R^2	Accuracy, precision, recall, F1, ROC-AUC
Watch out for	Outliers inflate MSE and RMSE	Class imbalance makes accuracy misleading





POST-LECTURE QUIZ

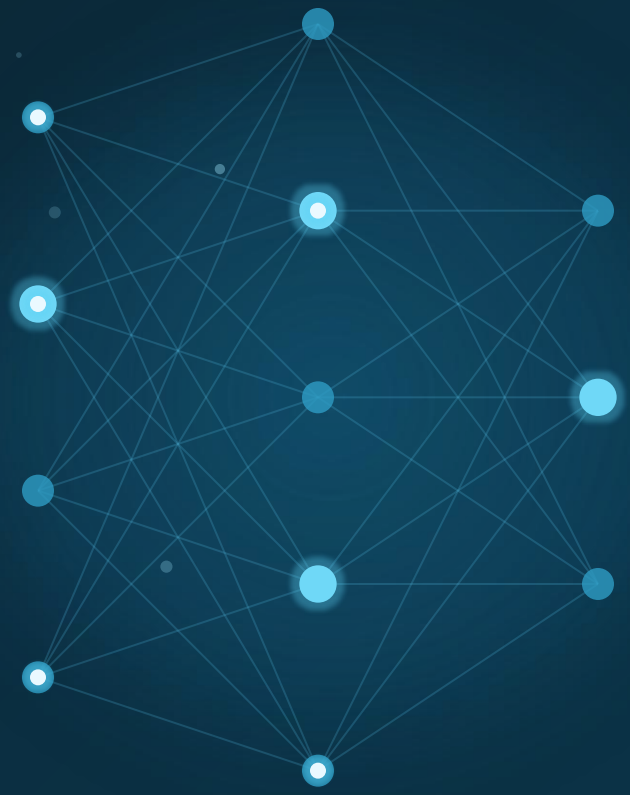
Introduction to Machine Learning

ff-quizzes.netlify.app/en/ml/quiz/2

DAY 1 · SECTION 1

Regression

Linear regression, loss and optimization, evaluation metrics



PRE-LECTURE QUIZ

Linear and Polynomial Regression

ff-quizzes.netlify.app/en/ml/quiz/13

Linear Regression

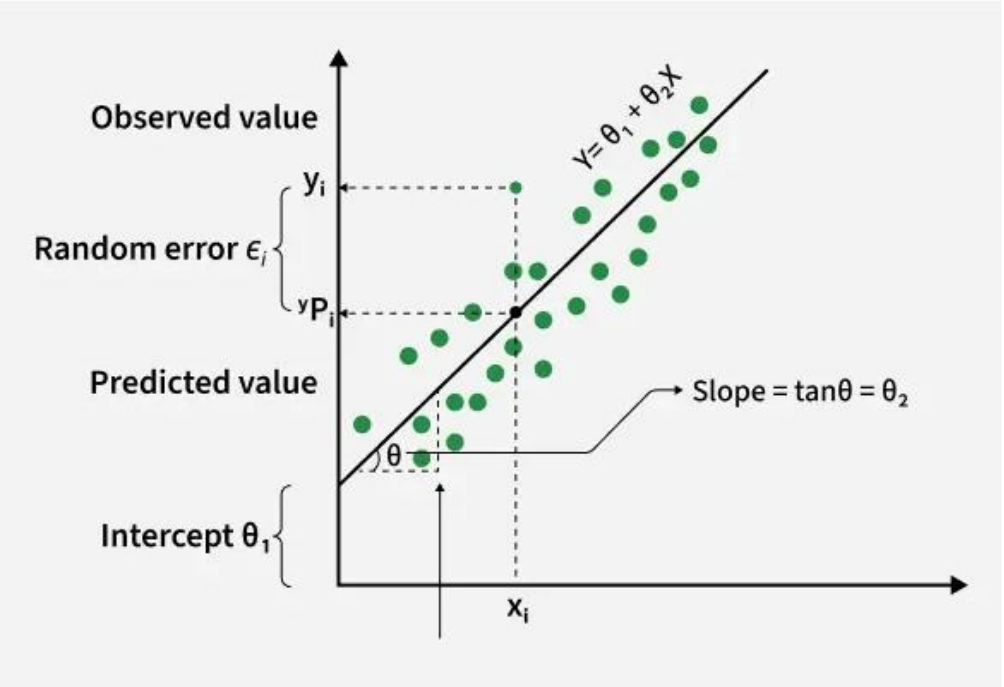
The model

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_n x_n$$

w_0 is the intercept; each weight w_i shows how much feature x_i moves the prediction.

Example: $\text{Price (\$K)} = 40 + 2.2 \times \text{Size (m}^2\text{)}$.

A 120 m^2 house is predicted at \$304K.



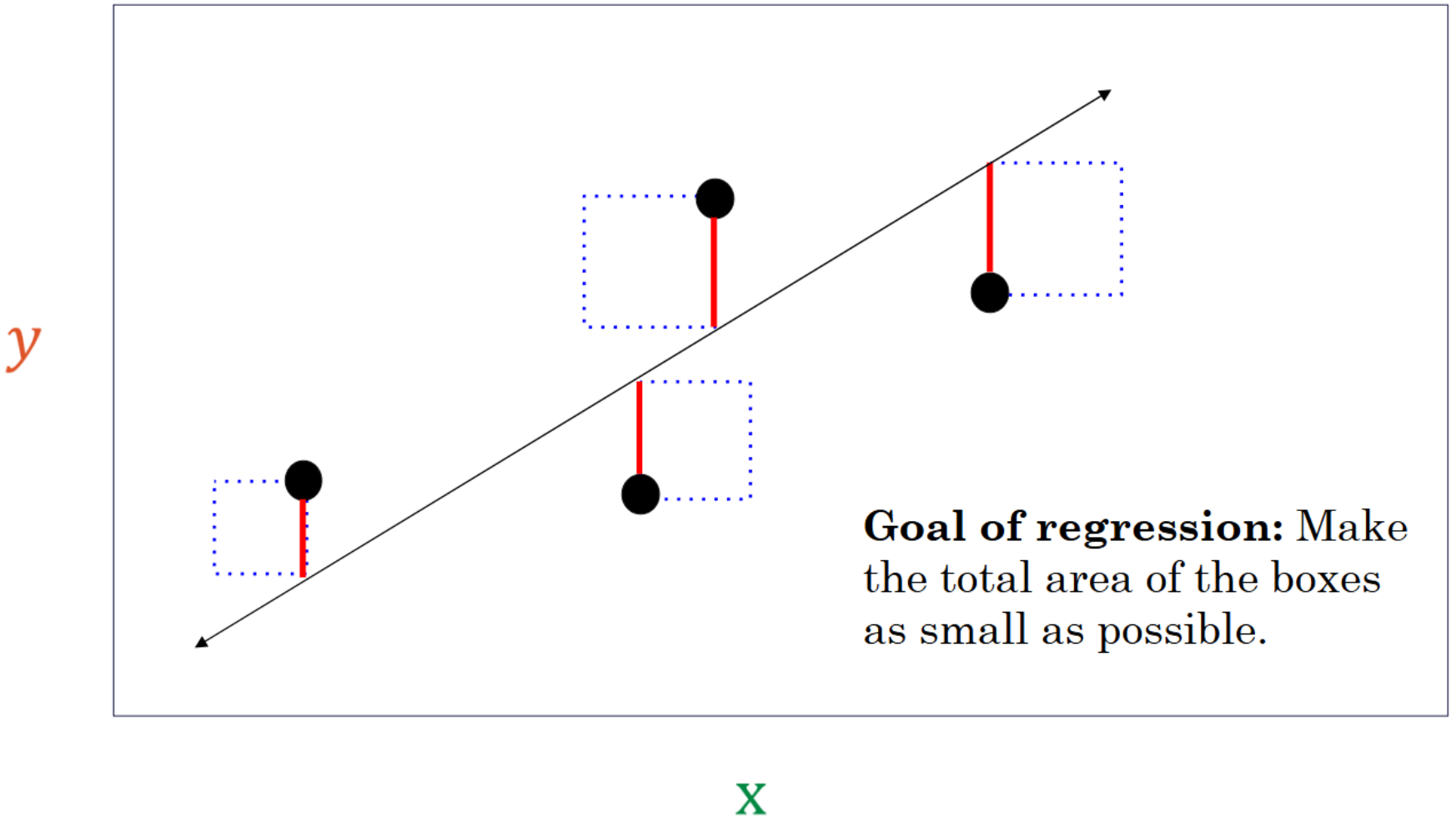
Training

Find the weights that minimize the squared error between the predictions $\hat{y}$ and the true values y.

Strengths and limits

- Fast, simple and easy to interpret
- Assumes a linear relationship; sensitive to outliers

Visualizing the Sum of Squared Error of Regression Model



Slide taken from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Regression Evaluation - Metrics

Mean Squared Error (MSE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2\right)$
Root Mean Squared Error (RMSE)	$\sqrt{\frac{1}{N}\left(\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2\right)}$
R-Squared (R^2) Score	$1 - \frac{\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2}{\sum_{n=1}^{N}(t_n - \bar{t_n})^2}$
Mean Absolute Error (MAE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\lvert t_n - y(\mathrm{x}_n, w)\rvert\right)$
Mean Absolute Percentage Error (MAPE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\frac{t_n - y(\mathrm{x}_n, w)}{t_n}\right) \times 100$

Slide taken from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Polynomial Regression

The model

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d$$

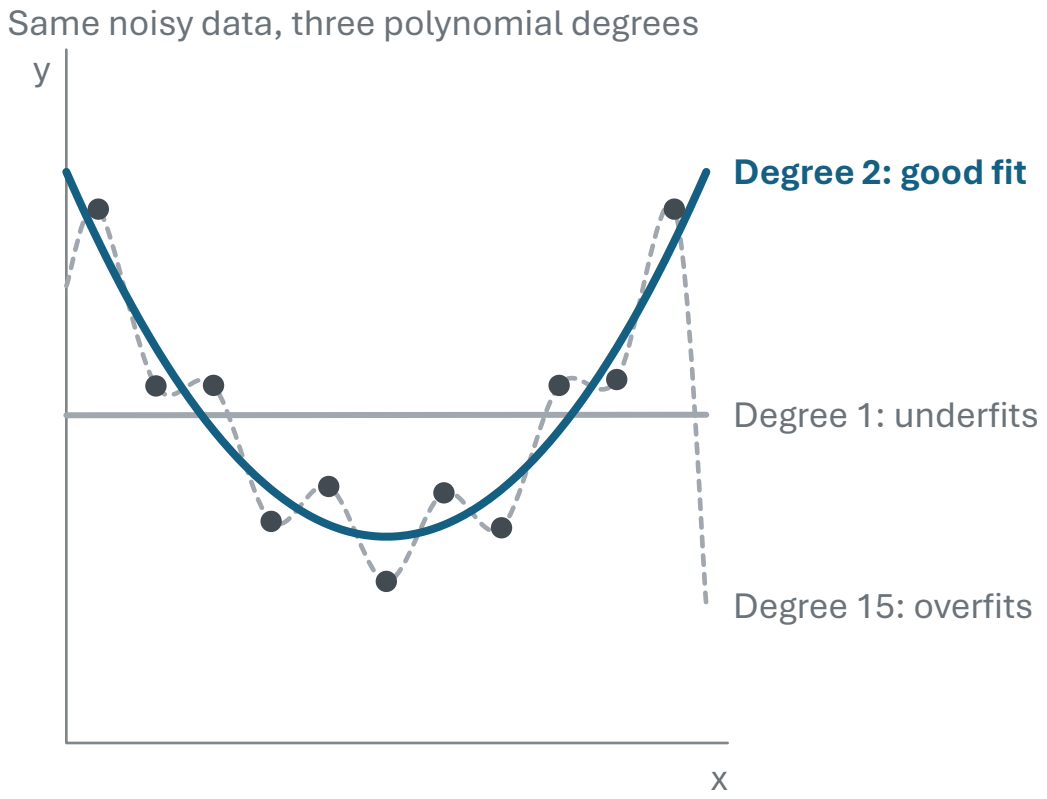
Powers of x become new features. The model stays linear in the weights, so least squares still fits it.

Choosing the degree d

- **Too low:** underfits and misses the curve
- **Too high:** overfits and chases the noise
- Pick d by validation error (cross-validation)

In scikit-learn

```
model = make_pipeline(PolynomialFeatures(degree=2),
    StandardScaler(), LinearRegression())
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```



POST-LECTURE QUIZ

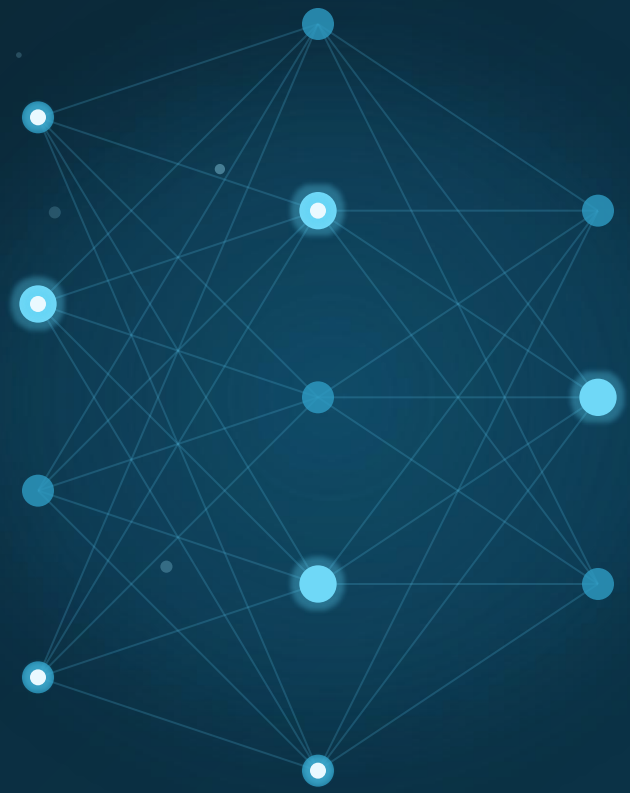
Linear and Polynomial Regression

ff-quizzes.netlify.app/en/ml/quiz/14

DAY 1 · SECTION 2

Classification

Logistic regression, confusion matrix, metrics, imbalanced data



PRE-LECTURE QUIZ

Classification 1

ff-quizzes.netlify.app/en/ml/quiz/19

PRE-LECTURE QUIZ

Logistic Regression

ff-quizzes.netlify.app/en/ml/quiz/15

Logistic Regression

The model

The sigmoid function turns any score into a probability between 0 and 1.

Training finds the weights that minimize **log loss (cross-entropy).**

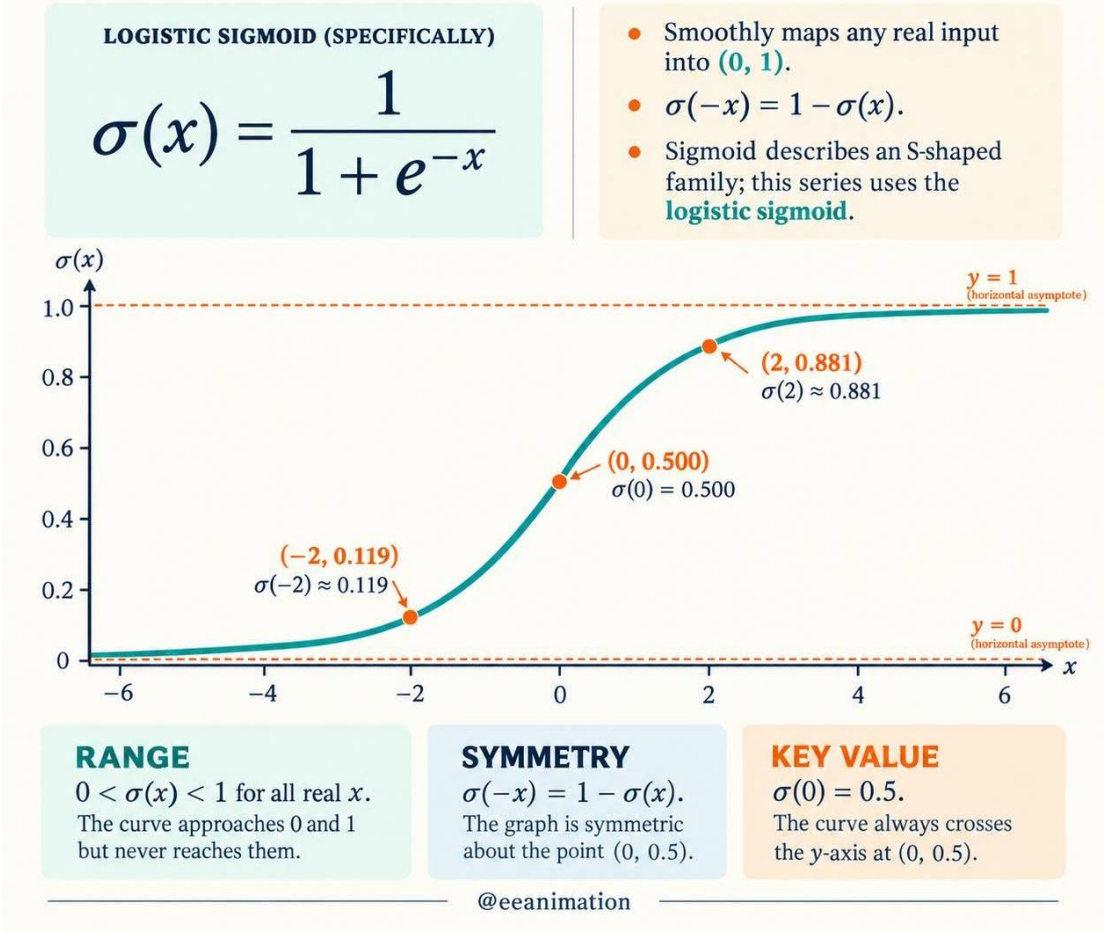
Decision rule

Predict class 1 when p ≥ 0.5. Moving this threshold trades precision for recall.

Example: a spam filter outputs p = 0.92 → spam; p = 0.10 → not spam.

Strengths and limits

- Fast, outputs probabilities, interpretable weights
- Draws a linear decision boundary



POST-LECTURE QUIZ

Logistic Regression

ff-quizzes.netlify.app/en/ml/quiz/16

Imbalanced Classification

Example: fraud detection on 1,000 transactions, of which 10 are fraud (1%)

99%

accuracy when always predicting “not fraud”

0 of 10

frauds caught: recall is 0%

Why accuracy misleads

- The model can ignore the minority class and still score high
- Few minority examples leave the model little to learn from

Where it appears

- Fraud detection
- Medical diagnosis
- Manufacturing defects
- Customer churn

Confusion Matrix

Example: a spam filter tested on 1,000 emails

	Predicted: spam	Predicted: not spam
Actual: spam	**TP = 85** spam caught	**FN = 15** spam missed
Actual: not spam	**FP = 30** real email flagged	**TN = 870** real email delivered

Accuracy = (TP + TN) / total = 955 / 1,000 = 95.5%

Every metric on the next slides comes from these four counts.

False positive (FP)

Type I error, a false alarm: a real email lands in the spam folder.

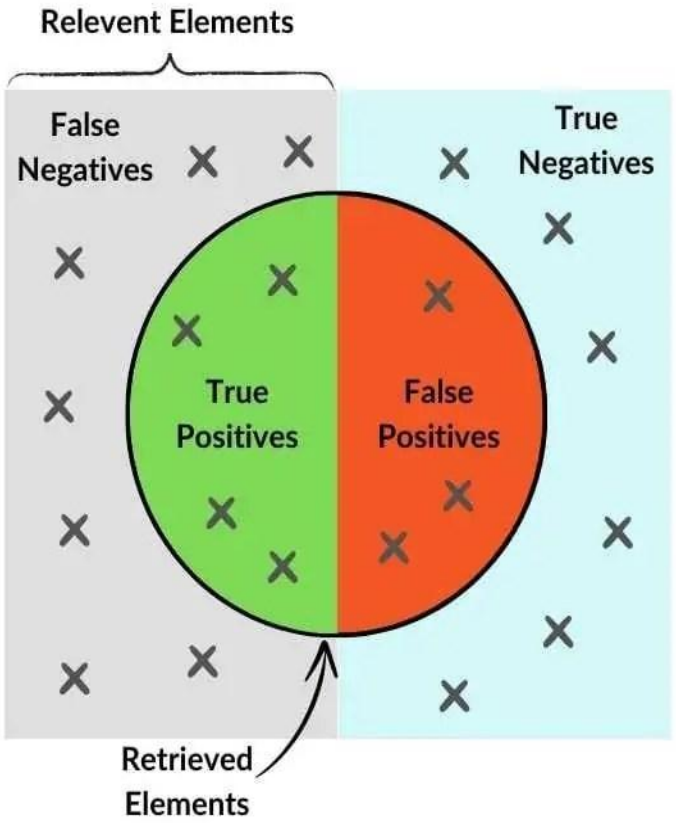
False negative (FN)

Type II error, a missed case: spam reaches the inbox.

Which error costs more?

It depends on the problem. In medical screening a missed disease (FN) is worse; for spam, losing a real email (FP) is worse.

Precision & Recall



How many retrieved elements are relevant?

Precision = $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$

How many relevant elements are retrieved?

Recall = $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$

F1 Score

Example: the same spam filter on 1,000 emails (TP = 85, FP = 30, FN = 15)

79%

F1 for the spam filter: precision 74%, recall 85%

2%

F1 if it flags 1 spam: precision 100%, recall 1%

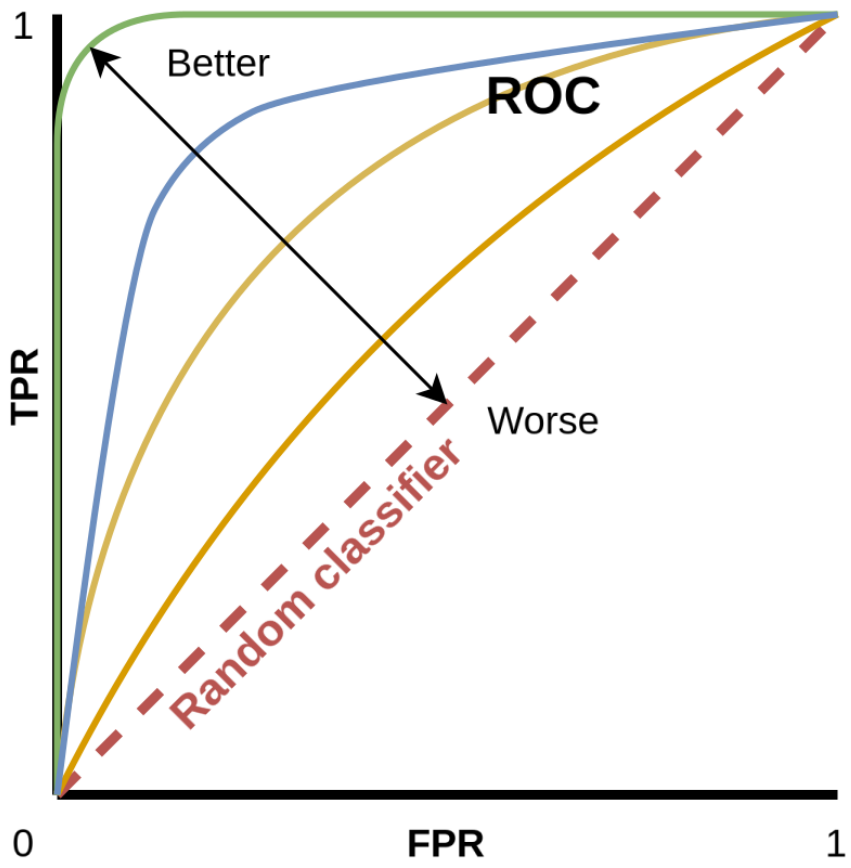
How it works

- F1 = 2 · P · R / (P + R)
- Harmonic mean: stays low unless precision and recall are both high
- Ignores TN, so the majority class cannot inflate it

When to use it

- Classes are imbalanced
- False alarms and misses both cost
- If one error costs more, track precision or recall instead

ROC Curve and AUC



From probability to class

Logistic regression outputs a probability; a threshold turns it into a class. Each threshold gives a different confusion matrix.

The ROC curve

True positive rate (TPR) (recall) = TP / (TP + FN) = 85%

False positive rate (FPR) = FP / (FP + TN) = 3.3%

The curve plots TPR against FPR across all thresholds.

AUC

Area under the curve: 1.0 is a perfect ranking, 0.5 is random guessing.

With heavy class imbalance, also check the precision-recall curve (PR-AUC).

Handling Imbalanced Data

Technique	How it works	Watch out
Class weights	Errors on the minority class cost more in the loss: class_weight='balanced'	Easiest first step; most scikit-learn models support it
Resampling	Oversample the minority class (SMOTE) or undersample the majority class	Resample the training data only, never validation or test
Threshold tuning	Move the decision threshold away from 0.5 to raise recall or precision	Choose the threshold on validation data
Better evaluation	Use stratified splits; report precision, recall, F1 and PR-AUC for the minority class	Accuracy alone hides the problem

Library: imbalanced-learn (imblearn) provides SMOTE and pipelines that resample only the training folds

POST-LECTURE QUIZ

Classification 1

ff-quizzes.netlify.app/en/ml/quiz/20

PRE-LECTURE QUIZ

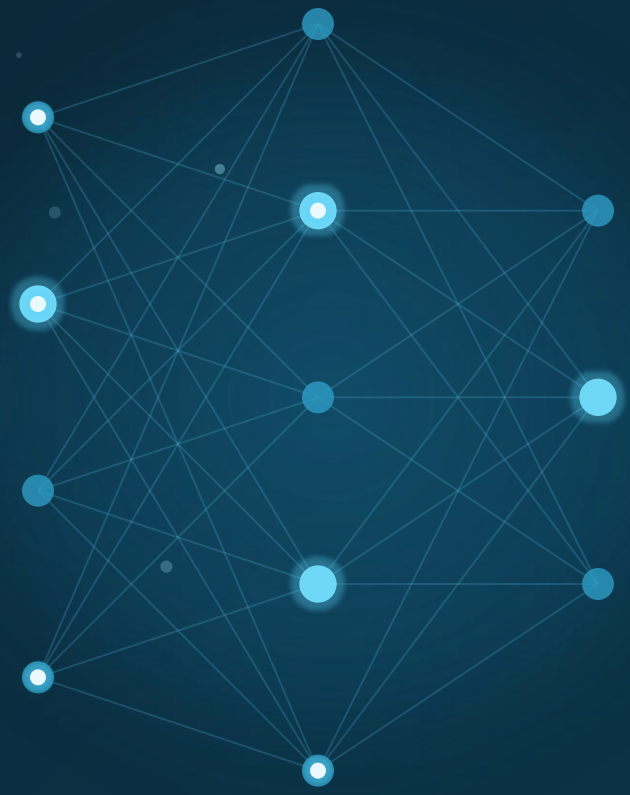
Classification 2

ff-quizzes.netlify.app/en/ml/quiz/21

DAY 1 · SECTION 3

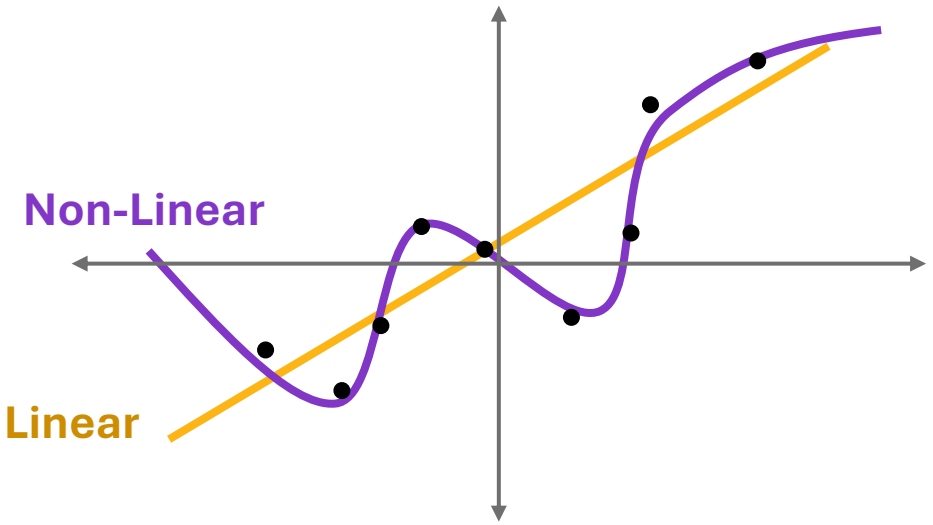
Regularization of Model Complexity

Overfitting, bias-variance trade-off, regularization, cross validation

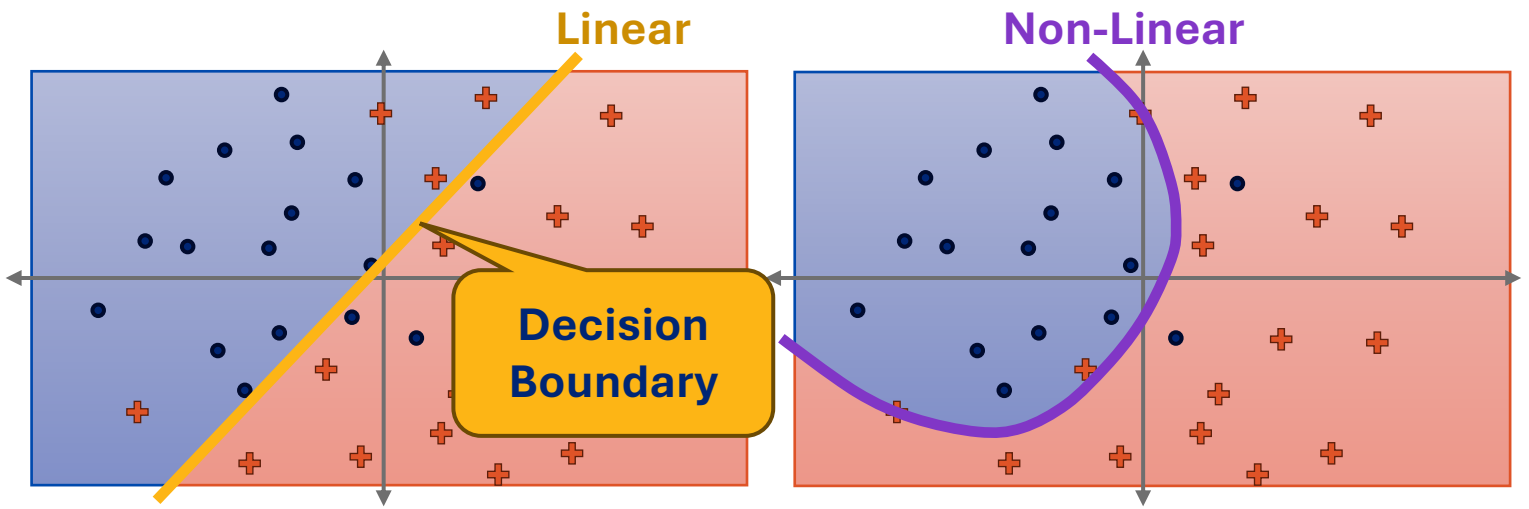


Complexity:
Linear Models vs Non-Linear Models

Regression



Classification (Decision Boundary)

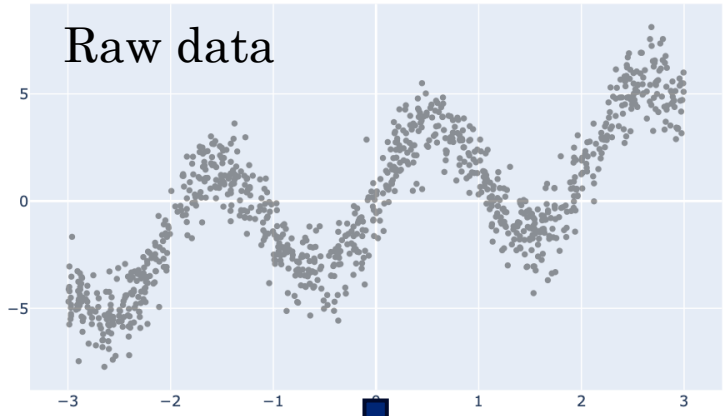


Nonlinear models are more expressive and can represent more complex relationships.

Are non-linear models better?

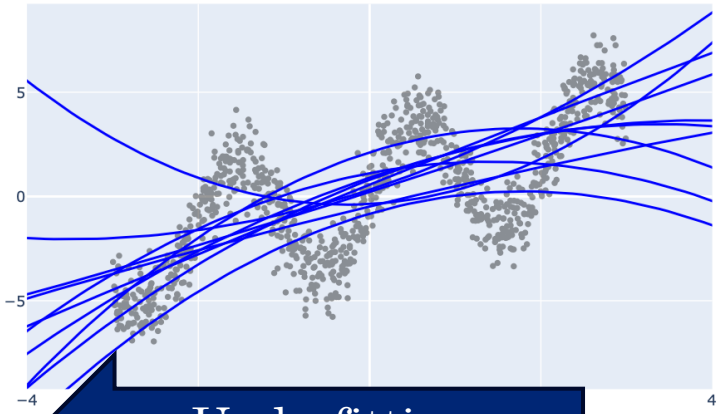
Slide taken from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Complexity and Overfitting

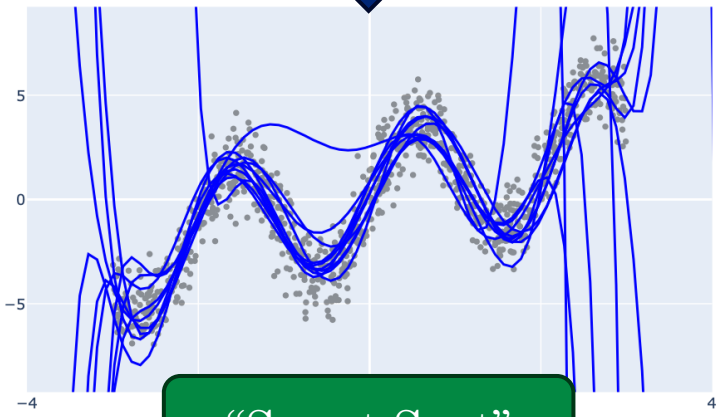


More complex isn't always better.

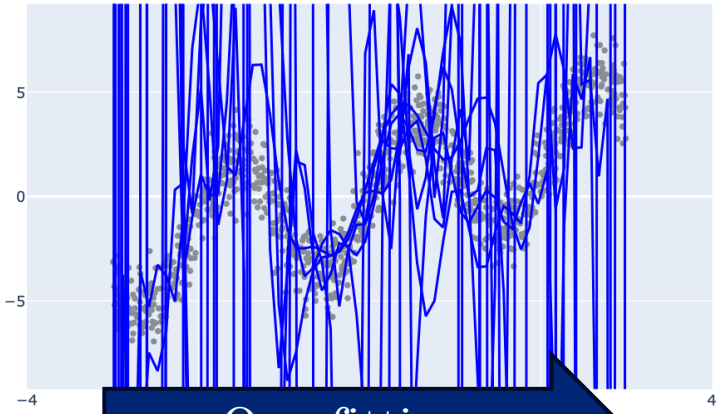
Fit models to samples of data



Underfitting



"Sweet Spot"



Overfitting

Slide taken from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Overfitting Problem

- Overfitting occurs when the model reflects noise or artifacts in training data, which do not generalize
- Small train error, at cost of test high error
- Hence, predictions of overfitting models caanot be trusted – but propoer ML evaluaten workflows should make it visible

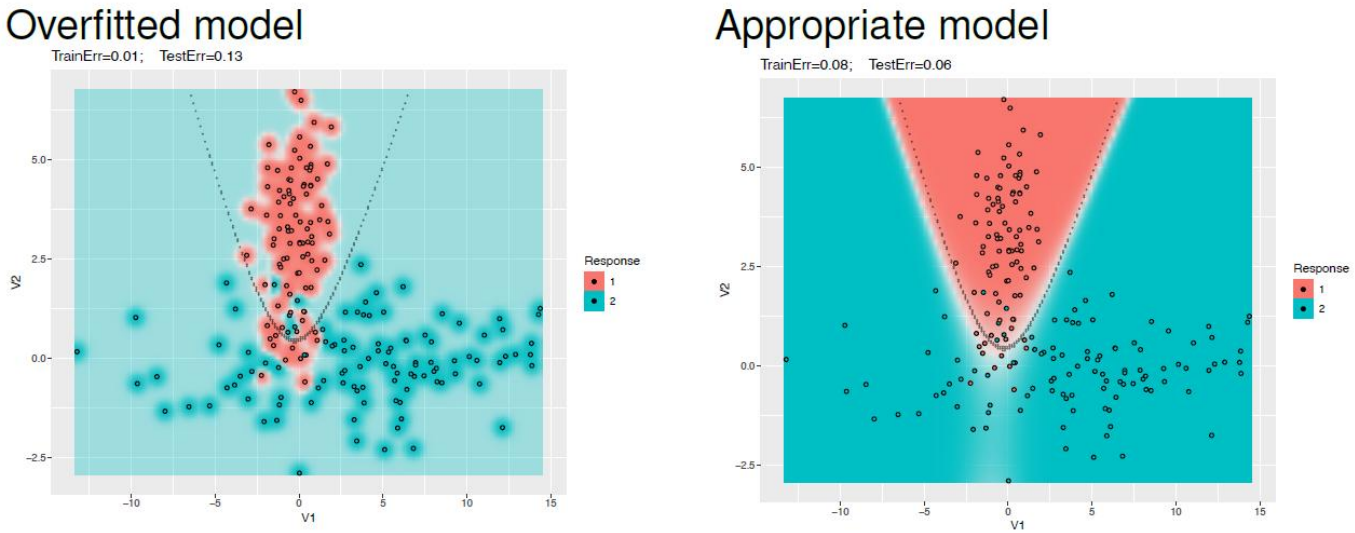
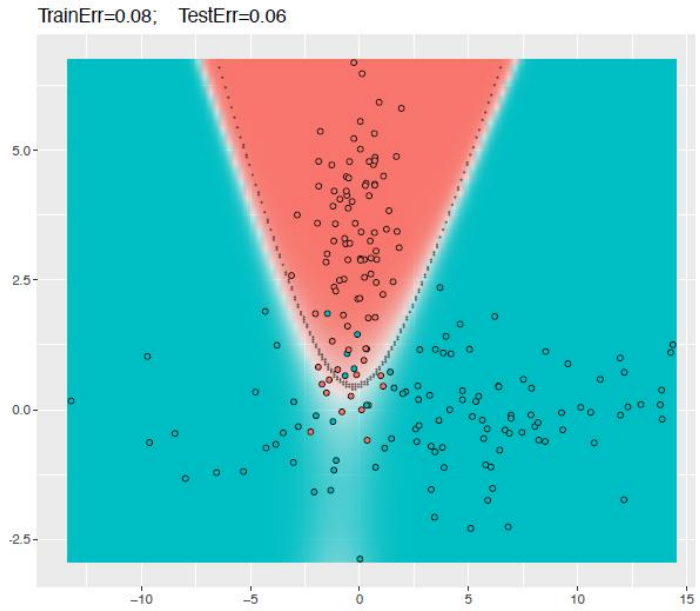


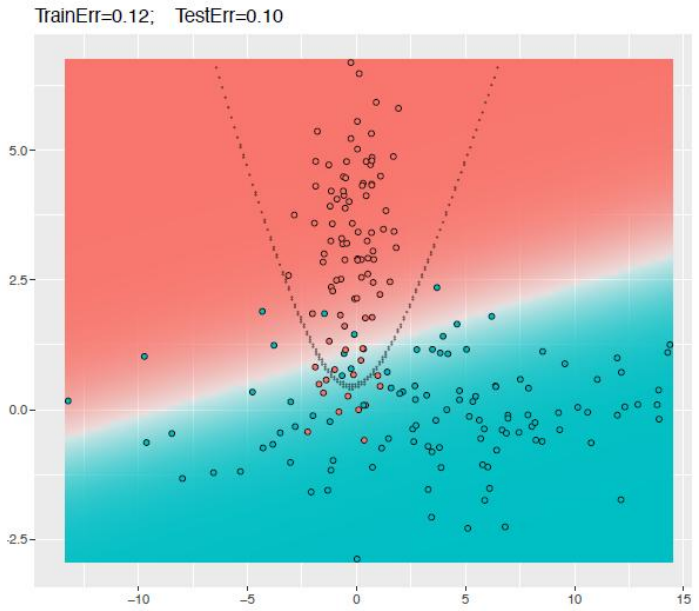
Image Source: Introduction to Machine Learning (I2ML)

Underfitting & Overfitting

Appropriate model



Underfitted model



Overfitted model

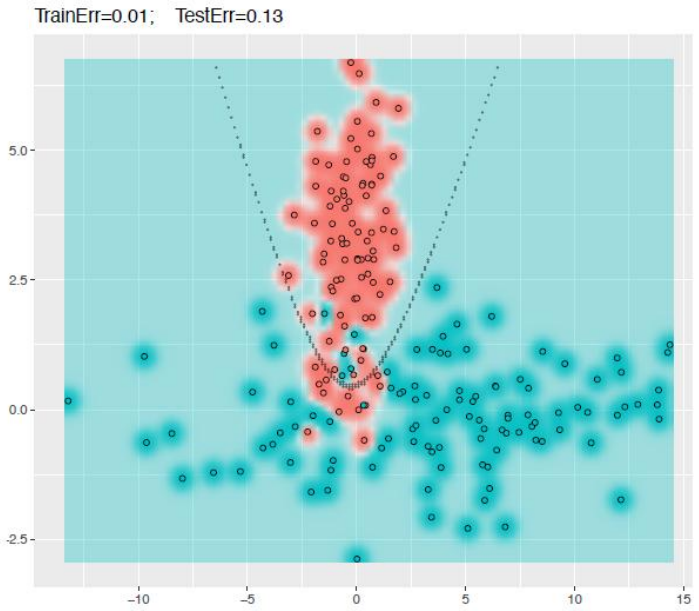


Image Source: Introduction to Machine Learning (I2ML)

Bias-Variance Trade-off

Expected error = Bias^2 + Variance + Irreducible noise

Making a model more complex lowers bias but raises variance; the best model balances the two.

	High bias: underfitting	High variance: overfitting
Meaning	The model is too simple and misses the pattern	The model memorizes noise in the training sample
Training error	High	Low
Validation error	High, close to training error	High, far above training error
How to fix	More features, a more flexible model, less regularization	More data, a simpler model, more regularization, bagging

Pick the complexity with the lowest validation error, the “sweet spot” on the previous slide.

Bias-Variance in Linear Regression

$$\hat{y} = \underbrace{\beta_0}_{\text{Bias part}} + \underbrace{\beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p}_{\text{Variance part}} + \underbrace{\varepsilon}_{\text{Noise}}$$

β_0, the intercept: bias

- A fixed baseline: the prediction when every x is 0. It does not react to the inputs.
- An intercept-only model predicts the training mean for everyone: **high bias, zero variance.**
- **Underfitting:** too few or too simple βx terms leave the work to β_0, so the model stays biased and misses the pattern.

$\beta_i x_i$ terms: variance

- Each coefficient is estimated from the training sample, so it shifts when the sample changes.
- More features fit the data closer (**bias falls**) but add estimates that can swing (**variance rises**).
- **Overfitting:** too many features or a high polynomial degree (x^2, x^3, …) let the coefficients fit noise in the sample.

Ridge and Lasso reduce overfitting: they shrink $\beta_1 \ldots \beta_p$ toward 0 but not β_0, trading a little bias for less variance.

Regularization

Add a penalty on large weights to the training loss:

Loss = Error(w) + λ · Reg(w)

L2 regularization (Ridge)

- $Reg(w) = \Sigma w_i^2$
- Shrinks all weights toward zero
- Keeps every feature

L1 regularization (Lasso)

- $Reg(w) = \Sigma |w_i|$
- Sets some weights exactly to zero
- Acts as feature selection

Choosing λ: λ = 0 means no penalty; a large λ underfits. Tune λ with **validation.**

POST-LECTURE QUIZ

Classification 2

ff-quizzes.netlify.app/en/ml/quiz/22

Training – Validation – Testing Data Splits



Training set

Learns the parameters

The model fits its weights (e.g., β coefficients) to these rows.

Validation set

Tunes the hyperparameters

Compare settings such as λ, tree depth or k, and keep the one with the lowest validation error.

Test set

Estimates generalization

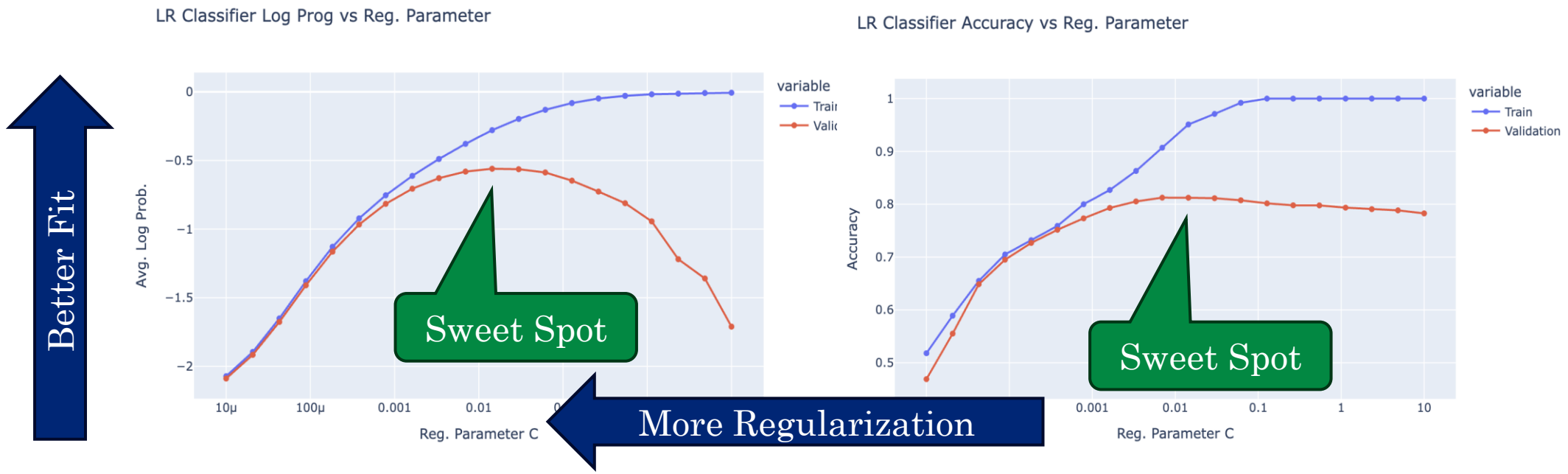
Used once, after every choice is made, to measure performance on unseen data.

Final model: after tuning and testing, retrain on all data (training + validation + test) with the chosen hyperparameters and deploy it to production.

Training vs Validation Accuracy

When tuning **regularization hyperparameters** against the validation dataset, it is common to see plots where

- training acc. **is greater than** validation accuracy
- acc. on the **training dataset continues to increase.**
- acc. on the **validation** dataset **increases** and **then decreases.**



Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

What if validation scores vary…

If a different random split changes the validation score a lot, one split is not enough.

Holdout split is enough

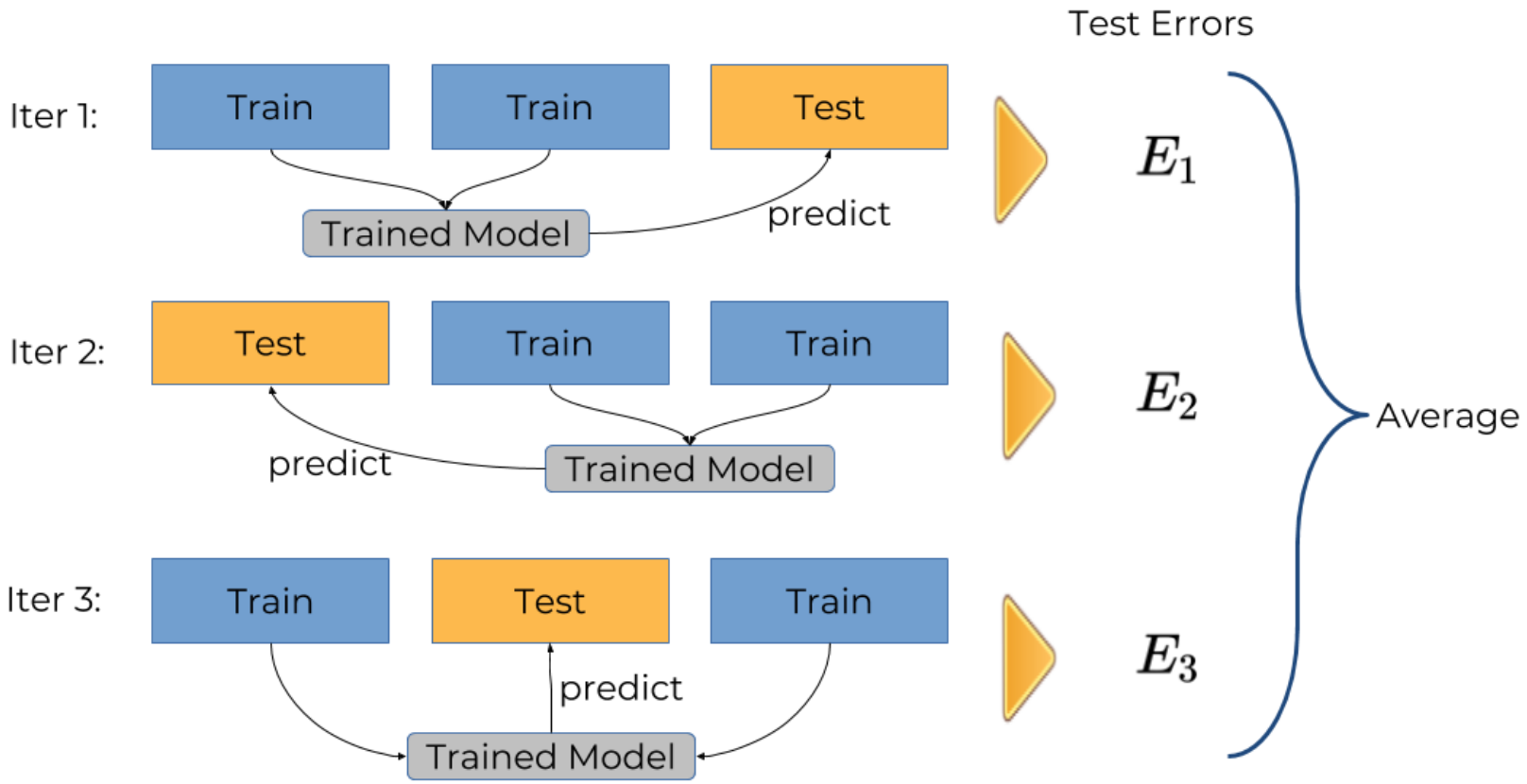
- **Large dataset:** each split is representative, so the score barely moves between splits.
- **Expensive training:** deep networks or large models, where training k times costs too much.
- **Time-ordered data:** train on the past and validate on a later period; never shuffle.

Use k-fold cross validation

- **Small or medium dataset:** one split can be lucky or unlucky; each row validates once.
- **Scores swing between splits:** average over k folds (k = 5 or 10) and report mean ± std.
- **Close model comparisons:** a gap of 1–2 points on one split may be noise.
- **Imbalanced classes:** stratified k-fold keeps the class ratio in every fold.

In both cases: keep the test set untouched during tuning; use it once, for the final estimate.

Cross Validation



Stratified Cross Validation

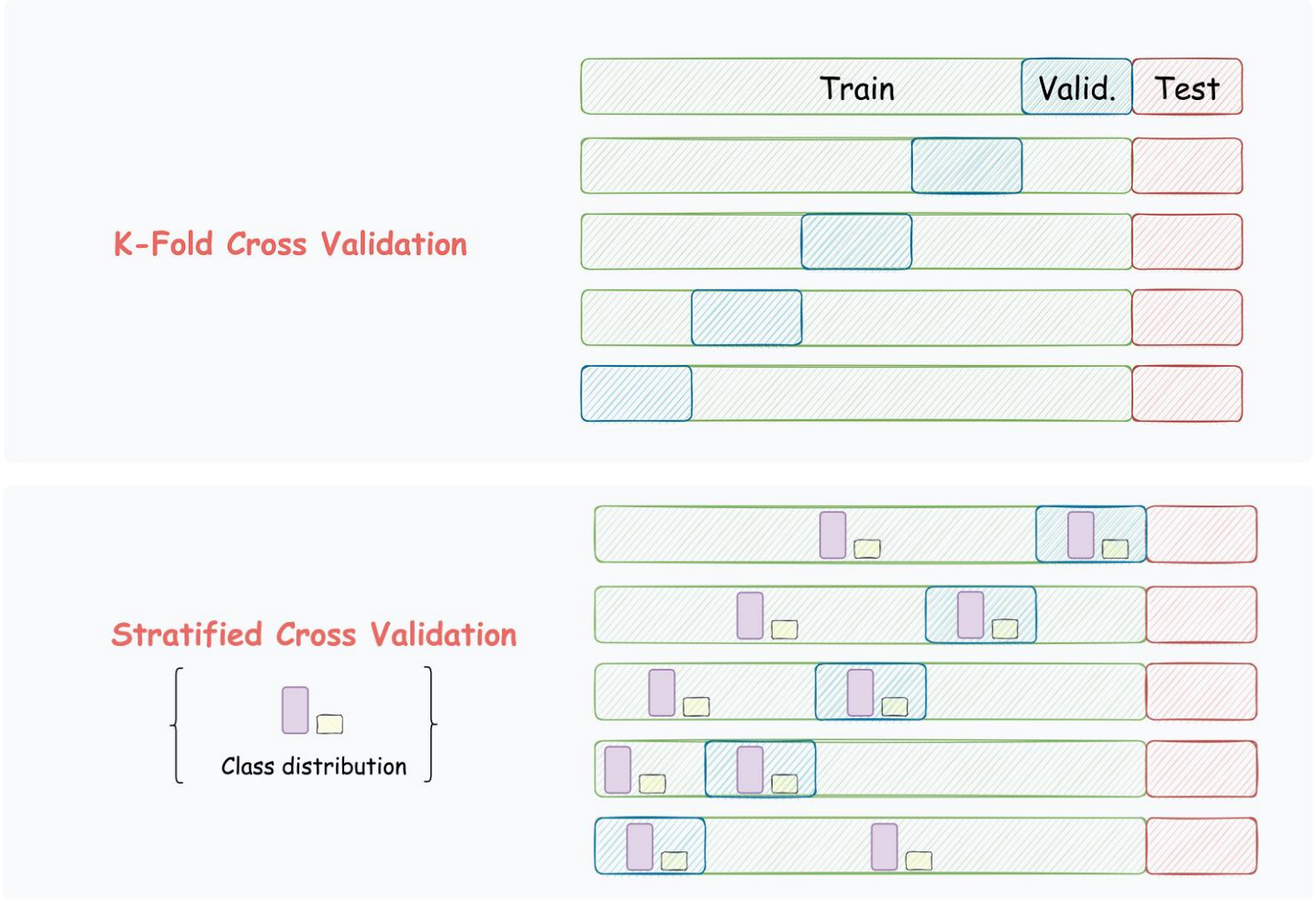
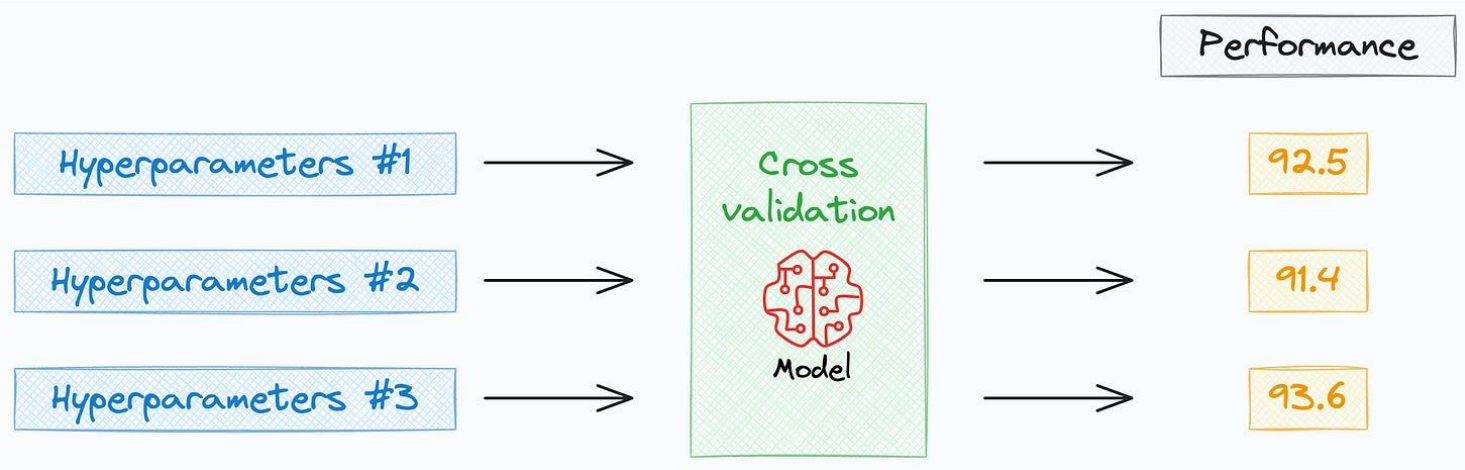


Image Source: https://blog.dailydoseofds.com/p/5-cross-validation-techniques-explained

What to Do After Cross Validation?



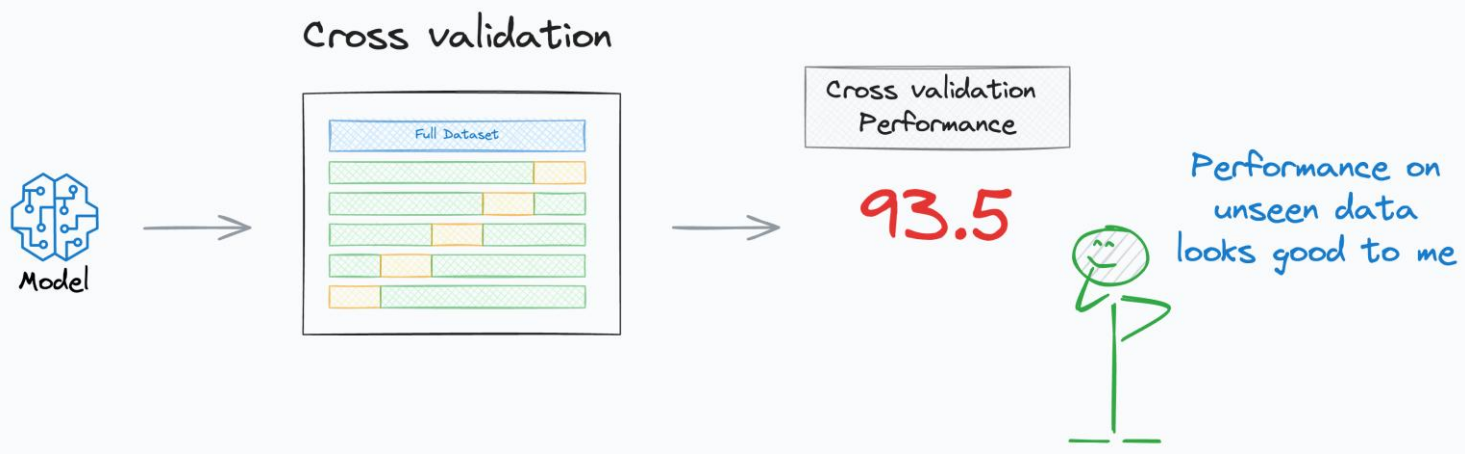


Image Source: https://blog.dailydoseofds.com/p/you-cross-validated-the-model-what

What really is a regularization hyperparameter?

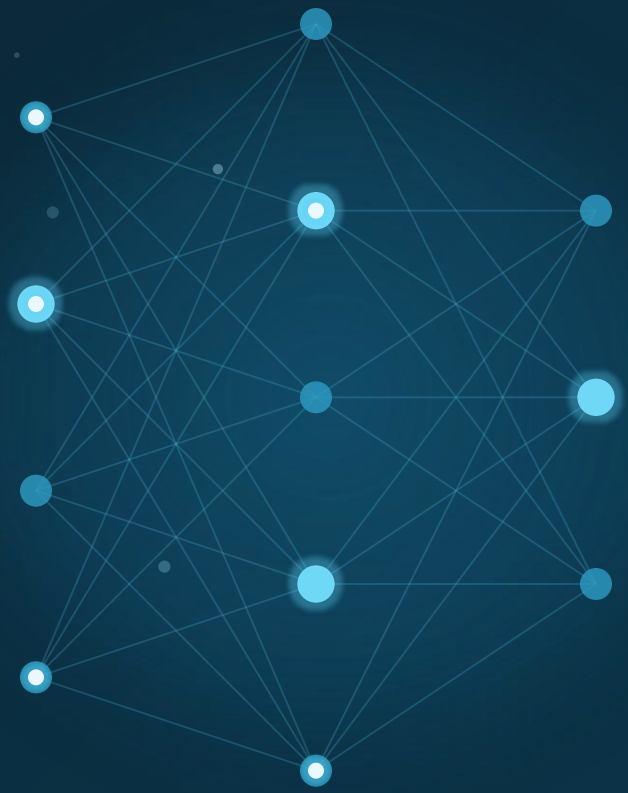
- Understand how they effect ML models
- What is regularization parameter(s) of each ML modelling approach?

DAY 1 · SECTION 4

Simple Models

✓ Linear Regression ✓ Logistic Regression

Naive Bayes, k-Nearest Neighbors, Decision Trees



PRE-LECTURE QUIZ

Classification 3

ff-quizzes.netlify.app/en/ml/quiz/23

Naive Bayes

Bayes' rule

P(class | x) = P(x | class) · P(class) / P(x)

Pick the class with the highest probability given the features x.

The naive assumption

Features are independent given the class, so the likelihood becomes a product:

$P(x \mid class) = P(x_1 \mid class) \times \ldots \times P(x_n \mid class)$

Variants

- **Gaussian NB**: continuous features
- **Multinomial NB**: word counts in text
- **Bernoulli NB**: yes/no features

Example: an email contains “free” and “winner”

Spam: 0.30 × 0.40 × 0.20 = 0.024

Not spam: 0.70 × 0.05 × 0.01 = 0.00035

Prior × P(“free” | class) × P(“winner” | class)

→ spam, with probability 0.024 / 0.02435 ≈ 99%

Strengths and limits

- Very fast; works with little data and many features, e.g. text
- Independence rarely holds; probabilities are often overconfident

k-Nearest Neighbors (kNN)

1

Choose k, the number of neighbors

2

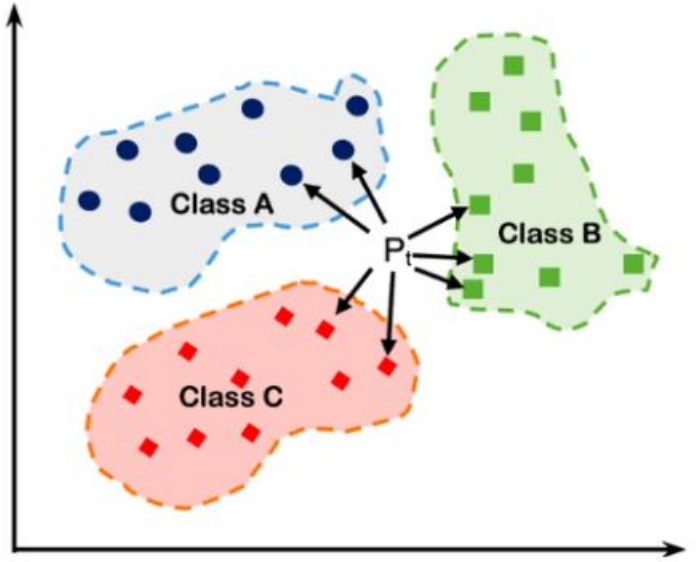
Measure the distance from the new point to every training point

3

Take the k closest points

4

Classification: majority vote.
Regression: average of their values



Choosing k

- Small k (k = 1) follows noise and overfits
- Large k smooths too much and underfits
- Pick k with cross validation; odd k avoids ties

Scale features first

- Otherwise size in m^2 dominates the number of rooms

Works well when

- Data is small and low-dimensional
- Class boundaries are irregular

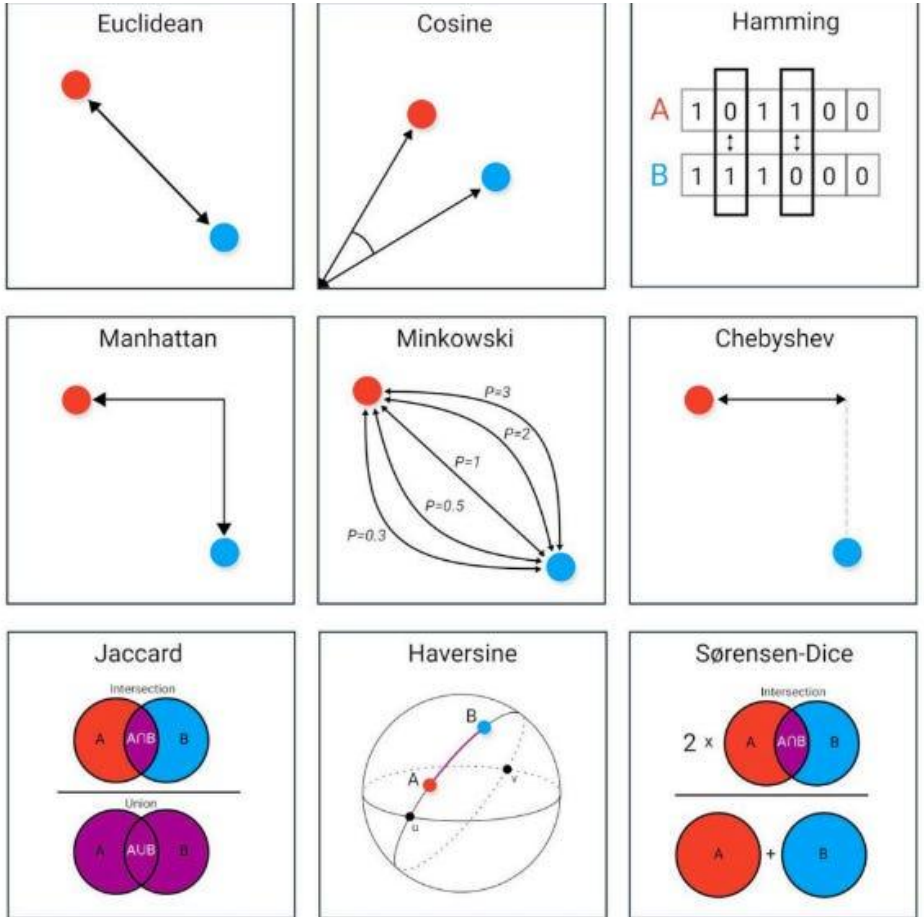
Struggles when

- Many features make distances meaningless
- Large data slows every prediction

Applications

- **Recommendations:** users with similar preferences
- **Anomaly detection,** filling missing values

Distance Metrics



Decision Trees

Questions about features

The tree asks yes/no questions such as “Income > 70k?” until it reaches a leaf, which gives the prediction.

Choosing a split

At each node, the tree tries every feature and threshold and keeps the split that makes the child nodes purest.

Gini impurity = $1 - \Sigma\, p_k^2$ **Entropy = $-\Sigma\, p_k \log_2 p_k$**

Example: a node with 50 spam and 50 not spam has

Gini = $1 - (0.5^2 + 0.5^2) = 0.5$.

A pure node has Gini = 0.

Regression trees

Splits minimize squared error; each leaf predicts the mean of its samples.

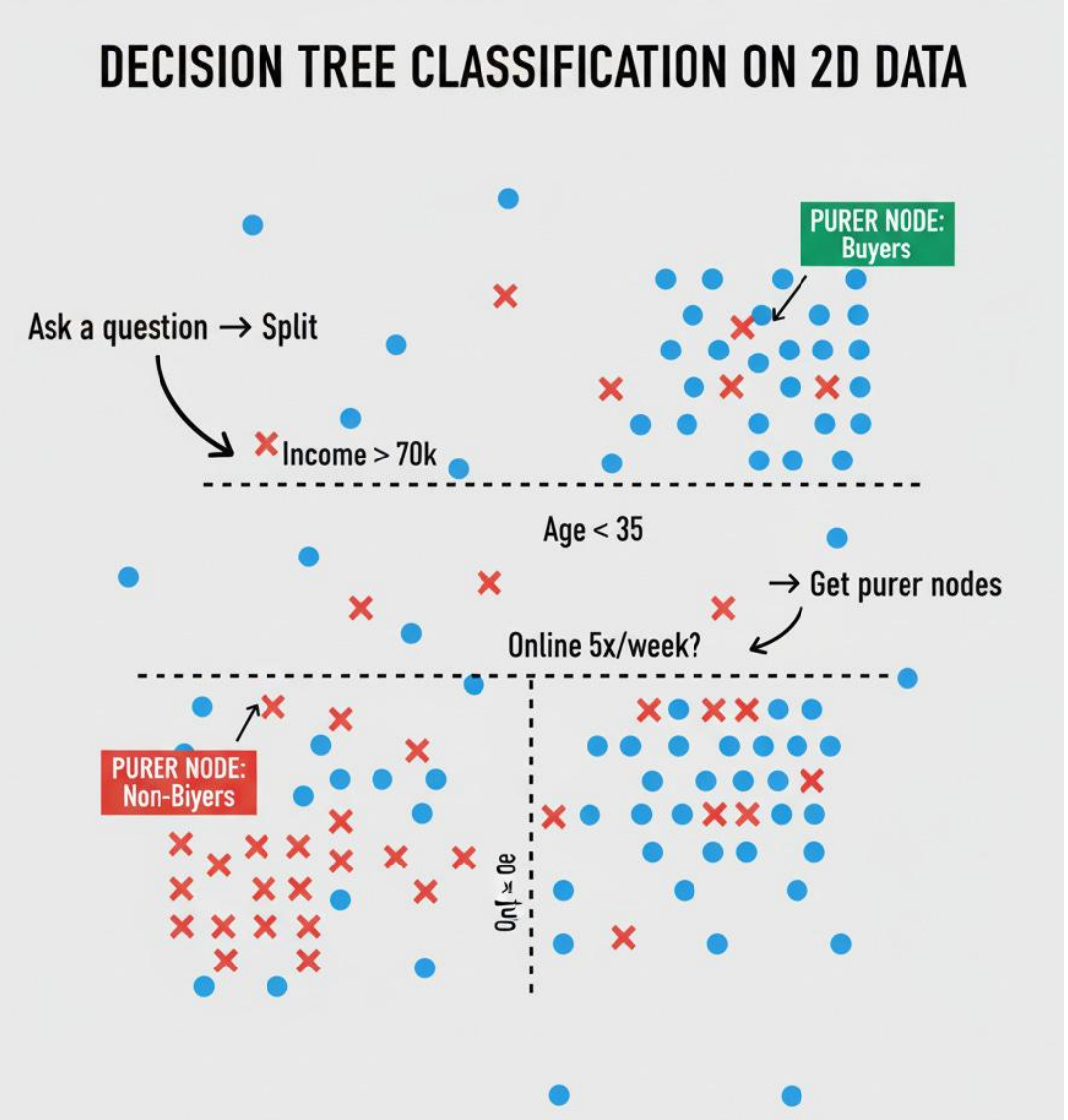


Image Source: Data Science Interview Guide – Part 4: Decision Trees

Decision Trees: Depth and Overfitting

A tree that grows until every leaf is pure memorizes the training data: **100% training accuracy, poor validation accuracy.**

Hyperparameter	What it controls
`max_depth`	How many questions in a row the tree can ask
`min_samples_split`	Samples a node needs before it may split
`min_samples_leaf`	Smallest number of samples allowed in a leaf
`ccp_alpha`	Pruning strength: larger values cut weak branches and give smaller trees

Strengths

- Easy to explain and visualize
- No scaling needed; handles mixed feature types

Limits

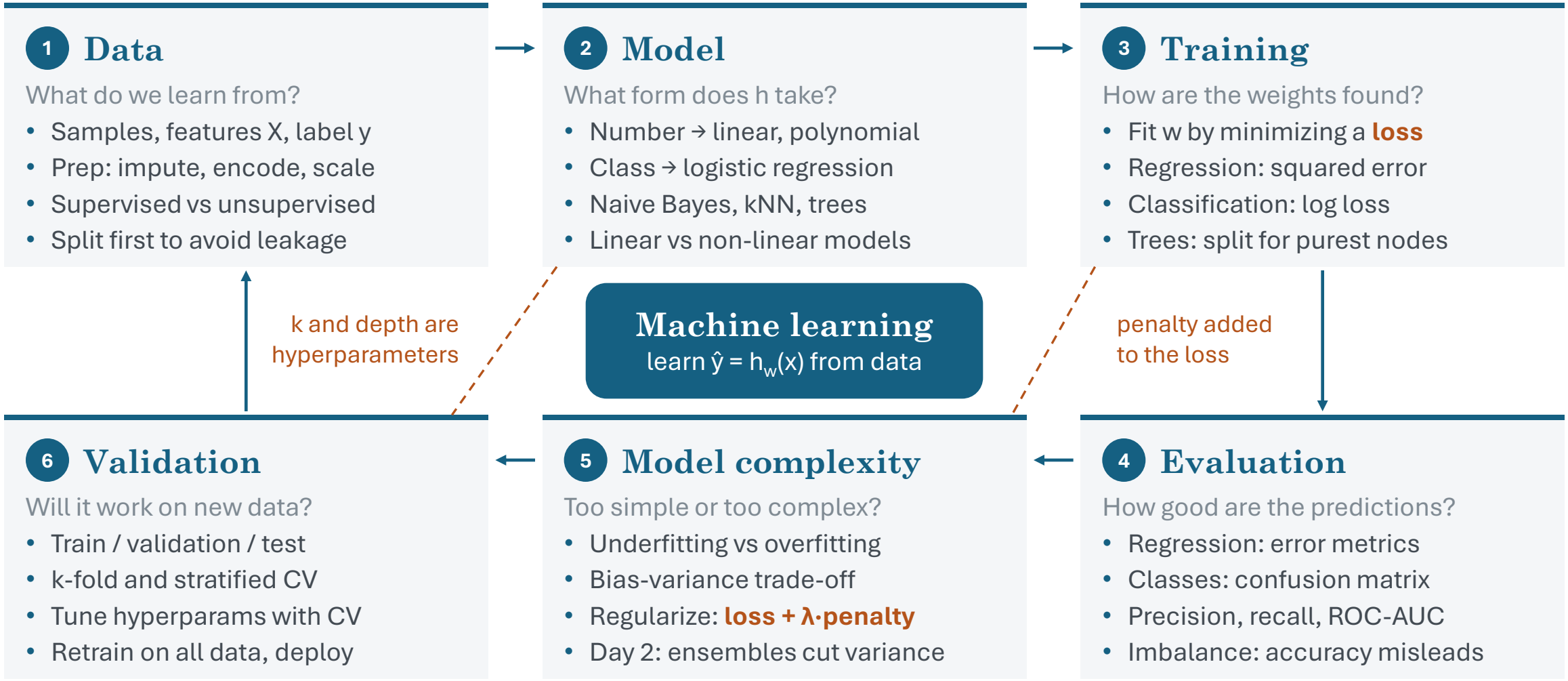
- High variance: small data changes give a different tree
- Ensembles of many trees fix this (Day 2)

POST-LECTURE QUIZ

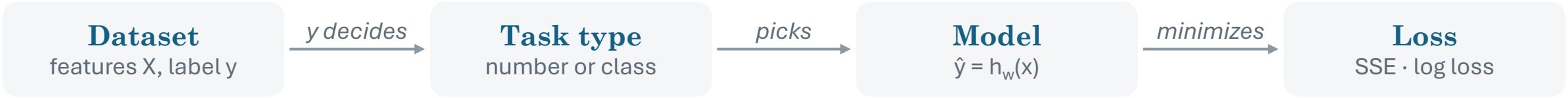
Classification 3

ff-quizzes.netlify.app/en/ml/quiz/24

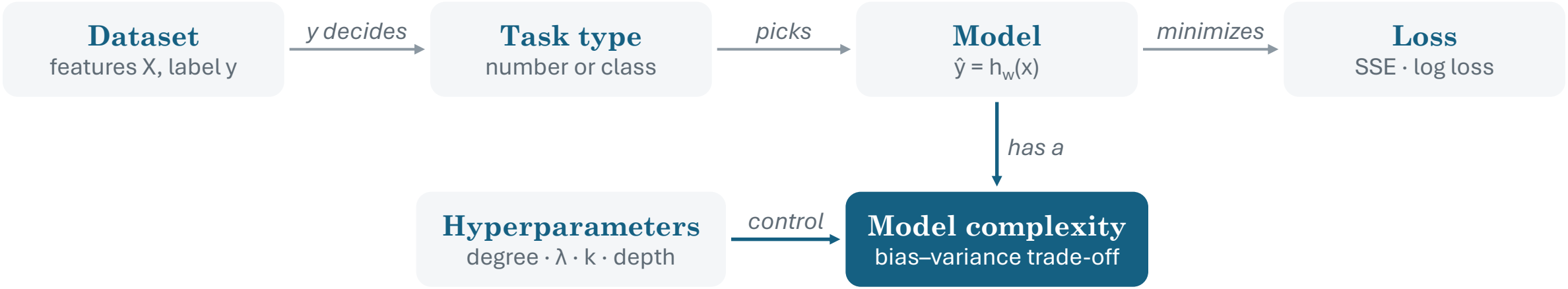
Day 1 Recap: How the Topics Connect



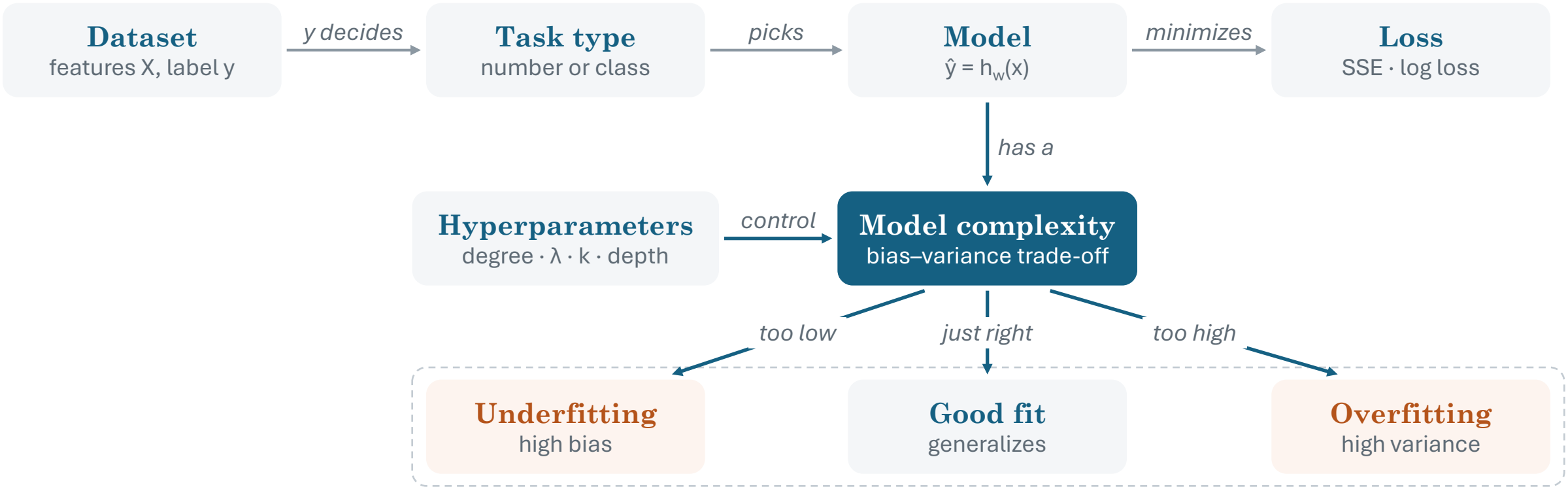
Day 1 Concept Map



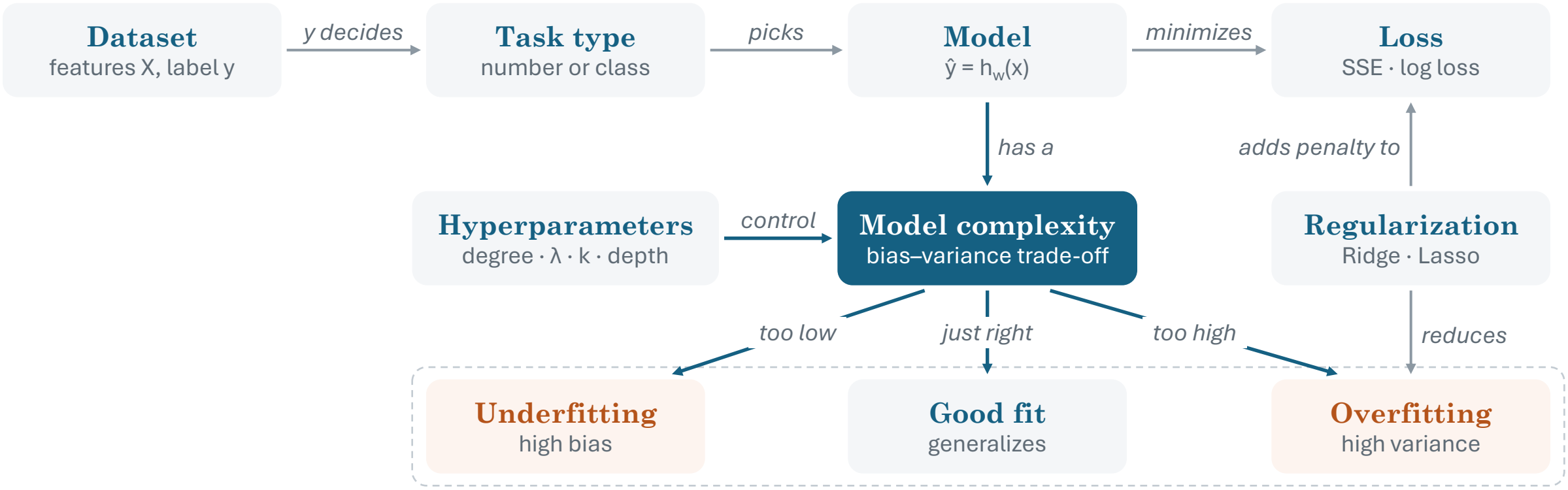
Day 1 Concept Map



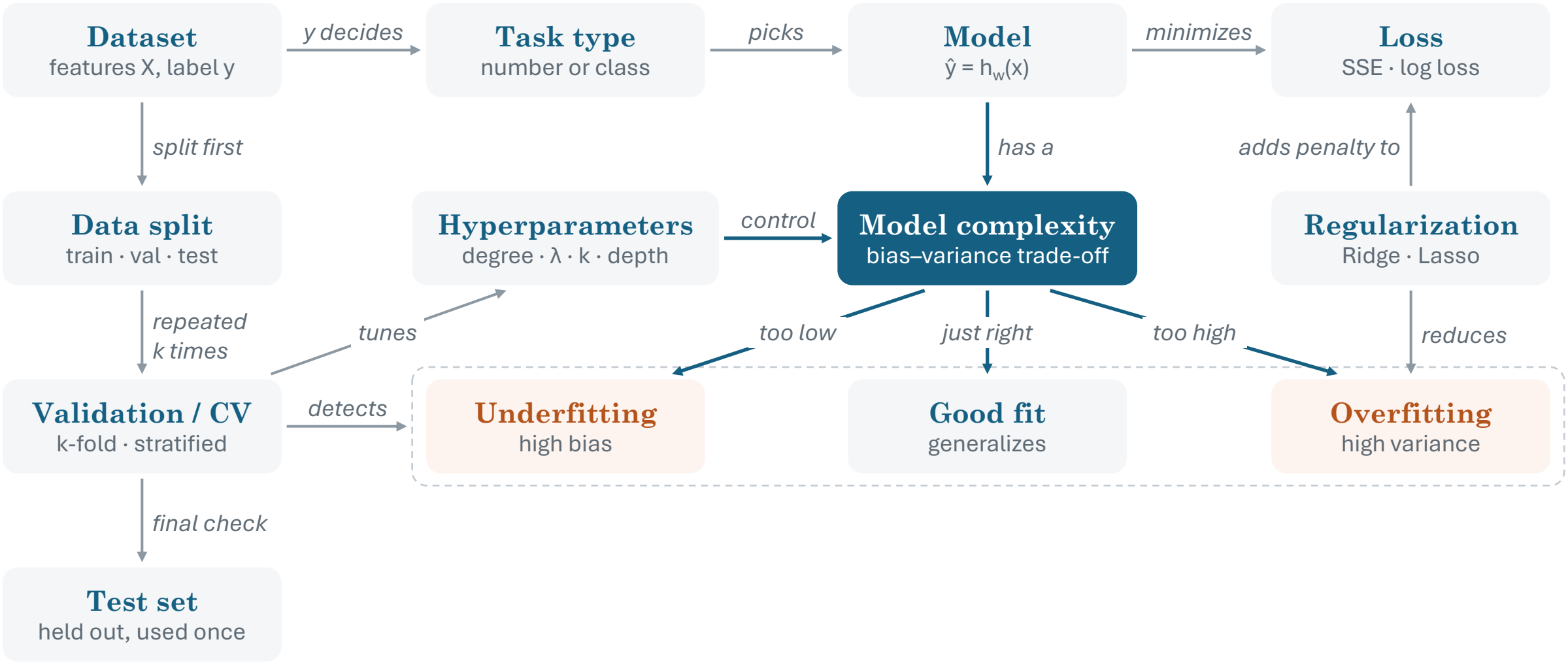
Day 1 Concept Map



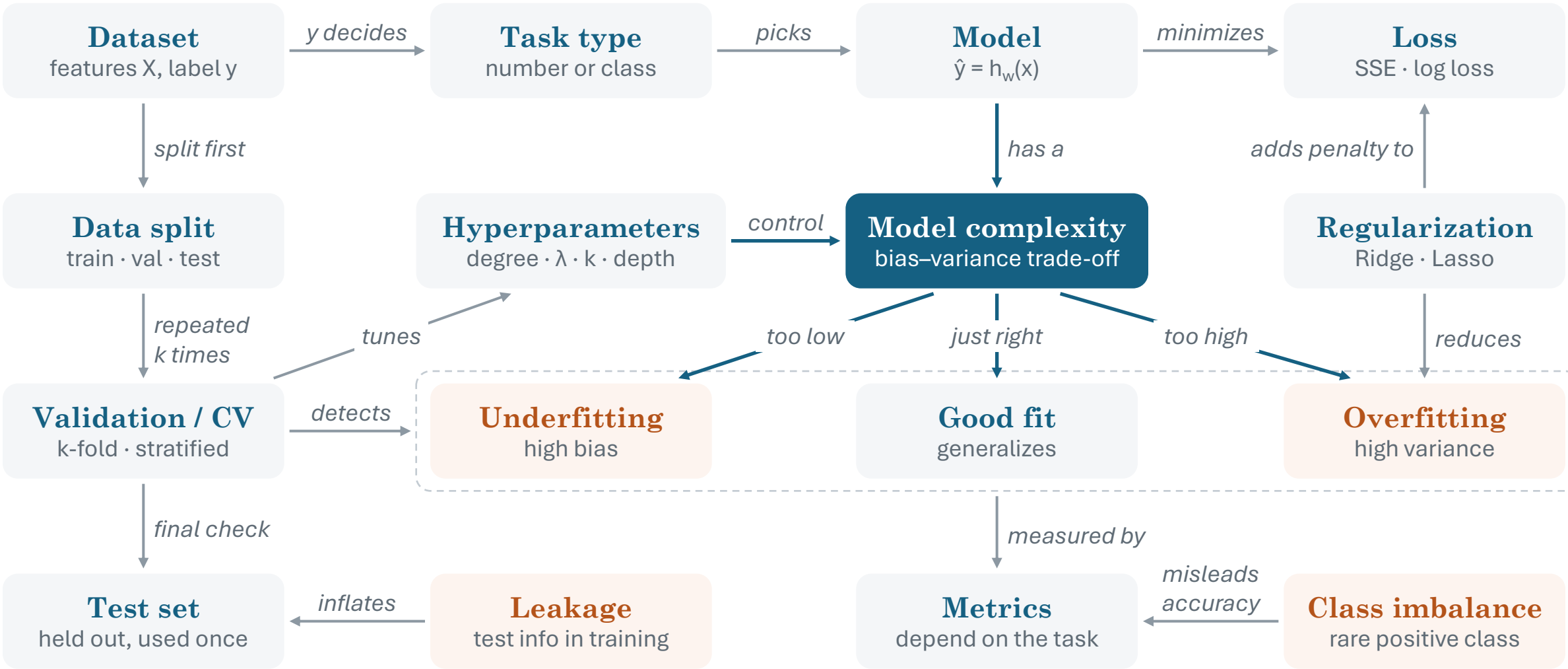
Day 1 Concept Map



Day 1 Concept Map



Day 1 Concept Map



Day 1 Summary

What we covered

- **Data:** split into train, validation and test; fit preprocessing on training data only
- **Regression:** predicts numbers; judge it with MAE, RMSE and R^2
- **Classification:** predicts classes; start from the confusion matrix, then precision, recall, F1 and ROC-AUC
- **Imbalanced data:** accuracy misleads; use class weights, resampling and threshold tuning
- **Overfitting:** more complex is not always better; control it with regularization and choose hyperparameters with cross validation
- **Models:** simple, fast baselines; Linear and Logistic Regression, Naive Bayes, kNN and Decision Trees

Day 2

- Ensemble models
- Deep learning (brief)
- Unsupervised learning
- Forecasting
- HPO with Optuna, tracking with MLflow
- Data science steps, ML lifecycle, interpretable ML